**Program :**

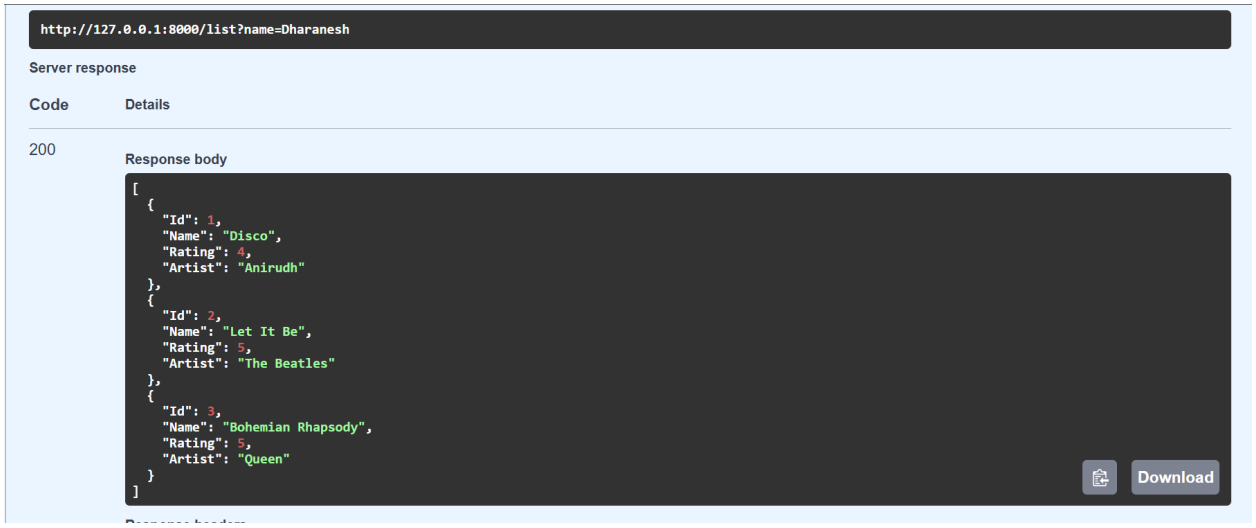
```
from fastapi import FastAPI, Depends, HTTPException, status, Query
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from pydantic import BaseModel
from typing import List, Annotated
from datetime import datetime, timedelta
import jwt
import uvicorn

SQLALCHEMY_DATABASE_URL = "sqlite:///./music.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False, "timeout": 30}  # prevent
locked DB
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

SECRET_KEY = "ahsdfpiugawiebf;iuwgefiubwjwhfljbadifuwieufbuiwbefiub"
ALGORITHM = "HS256"

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

class Music(Base):
    __tablename__ = "music"
    Id = Column(Integer, primary_key=True, index=True)
    Name = Column(String(50), nullable=False)
    Rating = Column(Integer, nullable=False)
    Artist = Column(String(50), nullable=False)

class User(Base):
    __tablename__ = "users"
    Id = Column(Integer, primary_key=True, index=True)
```

```
    Name = Column(String(50), unique=True, nullable=False)

Base.metadata.create_all(bind=engine)


class MusicSchema(BaseModel):
    Id: int
    Name: str
    Rating: int
    Artist: str

    class Config:
        from_attributes = True

class UserSchema(BaseModel):
    Id: int
    Name: str

    class Config:
        from_attributes = True

class UserCreateSchema(BaseModel):
    Name: str

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def create_access_token(data: dict, expires_delta: timedelta | None =
None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=30))
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

```python
async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_exception
        return username
    except jwt.PyJWTError:
        raise credentials_exception


@app.post("/register")
def register_user(user: UserCreateSchema, db: Session = Depends(get_db)):
    existing_user = db.query(User).filter_by(Name=user.Name).first()
    if existing_user:
        raise HTTPException(status_code=400, detail="Username already
exists")
    new_user = User(Name=user.Name)
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return {"message": "User registered successfully"}

@app.post("/login")
def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()], db:
Session = Depends(get_db)):
    user = db.query(User).filter_by(Name=form_data.username).first()
    if not user:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    token = create_access_token({"sub": user.Name})
    return {"access_token": token, "token_type": "bearer"}


@app.post("/add_song")
```

```python
def add_song(song: MusicSchema, db: Session = Depends(get_db)):
    new_song = Music(Name=song.Name, Rating=song.Rating,
Artist=song.Artist)
    db.add(new_song)
    db.commit()
    db.refresh(new_song)
    return {"message": "Song added successfully"}

@app.get("/list", response_model=List[MusicSchema])
def get_songs(name: str = Query(..., description="Username"), db: Session
= Depends(get_db)):
    user = db.query(User).filter_by(Name=name).first()
    if not user:
        raise HTTPException(status_code=401, detail="Not authorized")
    songs = db.query(Music).all()
    return songs

if __name__ == "__main__":
    uvicorn.run("MusicDb:app", host="127.0.0.1", port=8000, reload=True)
```

**Output :**



Request URL

```
http://127.0.0.1:8000/list?name=Dharaneshs
```

Server response

| Code | Details |
| --- | --- |
| 401<br>*Undocumented* | Error: Unauthorized<br>**Response body**<br>`{ "detail": "Not authorized" }`<br>Download<br>**Response headers**<br>`content-length: 27`<br>`content-type: application/json`<br>`date: Wed,20 Aug 2025 03:54:51 GMT`<br>`server: uvicorn` |